

SAECHAM: Secure and Efficient Lightweight Block Cipher CHAM Variant

Myoungsu Shin^{ID}, Hanbeom Shin^{ID}, Insung Kim^{ID}, Sunyeop Kim^{ID}, Dongjae Lee^{ID},
Deukjo Hong^{ID}, Jaechul Sung^{ID}, and Seokhie Hong^{ID}

Abstract—The addition, rotation, XOR (ARX) structure, which comprises three fundamental operations—Addition, Rotation, and XOR—makes it well-suited for lightweight cryptography. To design a secure and efficient ARX cipher, it is necessary to find the optimal structure by properly combining the order, number of operations, and rotation amounts. CHAM64 is an ARX block cipher with a 64-bit block size, which is proposed as an attempt to enhance the lightweight characteristics of LEA. In this article, we present secure and efficient CHAM (SAECHAM), a variant of CHAM64 with a rearranged order of operations and adjusted rotation amounts. By changing the order of the operations in CHAM64, six different CHAM-like structures can be created. We propose the properties that can be eliminated in the implementation process depending on the rotation amount in each structure. To improve suitability for constrained environments, such as 8-bit and 16-bit microcontrollers, we reduce the search space for rotation amounts and analyze the number of instructions. Using an SMT solver-based automatic search method, we analyze the security of 62 CHAM64 variants through differential and linear analysis. Among them, we find four variants with equal or better resistance to differential and linear cryptanalysis compared to CHAM64. As a result, we propose the variant with the fewest instructions among them as SAECHAM. Through software implementations on 8-bit AVR, 16-bit MSP430, 32-bit ARM Cortex-M3 and Cortex-M4 platforms, we demonstrate that SAECHAM is efficient in terms of encryption speed and also performs efficiently when implemented using SIMD operations in high-performance CPUs.

Index Terms—Addition, automatic search, AVR, CHAM, Cortex-M3, Cortex-M4, lightweight block cipher, MSP430, rotation, SAT/satisfiability modulo theories (SMT) solver, XOR (ARX) cipher.

I. INTRODUCTION

IN THE IoT environment, where countless devices are connected and exchange data, security has become increasingly important. This is especially true for IoT devices, which

are often limited in computational power and energy. These constraints make lightweight block ciphers essential. Designed for efficient data protection, lightweight block ciphers ensure fast processing speeds and low energy consumption, both critical for IoT applications.

Among lightweight block ciphers, addition, rotation, XOR (ARX)-based designs stand out due to their simple and efficient structure. By avoiding costly operations, such as multiplication, ARX ciphers provide strong security while maintaining high efficiency in both software and hardware implementations. As a result, ARX-based lightweight block ciphers are a practical solution to the security challenges in IoT systems.

CHAM is an ARX-based lightweight block cipher proposed by the National Security Research Institute (NSR) in South Korea and was presented at ICISC 2017 [1]. A revised version with additional security analysis, which increases the number of rounds, was presented at ICISC 2019 [2]. CHAM is designed to achieve high efficiency in low-resource micro controller unit (MCU) environments, such as AVR and MSP430. CHAM is a representative ARX cipher and is covered in various studies on the security analysis of ARX ciphers [3], [4], [5], [6], [7]. Similar to LEA [8], CHAM emphasizes encryption speed over a balanced approach to encryption and decryption speeds, aligning with the frequent usage of encryption in block cipher modes under ISO/IEC standards. Furthermore, CHAM has been utilized alongside SPECK [9] in the authenticated encryption scheme COMET [10], which advanced to the second round of the NIST Lightweight Cryptography standardization competition.

A. Related Works

When designing a block cipher, the choice of structure and components can significantly affect its efficiency and security. Therefore, block cipher designers configure structures and components with both security and efficiency in mind. GIFT [12] is an substitution permutation network (SPN)-based block cipher that uses the bad output must go to good input (BOGI) strategy for selecting its S-box and permutation. At EUROCRYPT 2022, L. Sun et al. [13] proposed a variant of GIFT-64 with the same BOGI design principles as GIFT-64, but more efficient and secure. The authors identified equivalence classes to reduce the search space for GIFT-64 variants, which are designed with the BOGI strategy, and used a SAT-based automatic search method. As a result, 24 GIFT-64 variants were discovered that showed better resistance to

Received 20 January 2025; revised 4 March 2025; accepted 9 May 2025. Date of publication 13 May 2025; date of current version 25 July 2025. This work was supported by the Institute of Information and Communications Technology Planning and Evaluation (IITP) Grant funded by the Korea Government (MSIT) under Grant RS-2023-00229028. (Corresponding author: Seokhie Hong.)

Myoungsu Shin, Hanbeom Shin, Insung Kim, Sunyeop Kim, and Seokhie Hong are with the Institute of Cyber Security and Privacy (ICSP), Korea University, Seoul 02841, Republic of Korea (e-mail: shhong@korea.ac.kr).

Dongjae Lee is with the Department of Convergence Security, Kangwon National University, Chuncheon 24341, Gangwon, Republic of Korea.

Deukjo Hong is with the Department of Computer Science and Artificial Intelligence, Jeonbuk National University, Jeonju 54896, Jeollabuk, Republic of Korea.

Jaechul Sung is with the Department of Mathematics, University of Seoul, Seoul 02504, Republic of Korea.

Digital Object Identifier 10.1109/IIOT.2025.3569746

differential cryptanalysis while maintaining similar resistance to linear cryptanalysis.

B. Motivation

Based on these insights from SPN-based designs, similar considerations can be applied to ARX ciphers. The ARX ciphers consist of various structures and the order of operations can vary widely. Even for simple structures, the order of operations and components can affect both security and efficiency. For example, adjusting the rotation amount in the round function can affect the security and alter efficiency on MCU. In addition, designing ARX ciphers without relying on table look-ups that are commonly used in S-box-based designs improves their resistance to side-channel attacks. CHAM is one of the simplest ARX ciphers for 8-bit AVR and 16-bit MSP430. We can easily observe the impact of changes to the rotation amount. In addition, CHAM64 has been shown in [14] that changing the order of operations results in differences in encryption speed on modern architectures.

C. Our Results

This article presents SAECHAM, a variant of CHAM64. We focus on optimizing encryption speed on 8-bit AVR and 16-bit MSP430 microcontrollers, which is one of the key design principles of CHAM. More specifically, we consider the rotation cost on 8-bit AVR and 16-bit MSP430, and SAECHAM has lower rotation costs than CHAM64. Furthermore, SAECHAM is more secure than CHAM64. SAECHAM has similar resistance to linear cryptanalysis as CHAM and provides an improved resistance to differential cryptanalysis.

The round function of CHAM has a simple operation structure. Even with this simplicity, security and efficiency can vary just by changing the order of operation within the structure. We analyze the rotation costs, which vary depending on the CHAM-like structures, and propose conditions under which specific rotations can be eliminated to optimize performance. As a result, we identify 62 variants that are more efficient than CHAM64 and perform linear and differential cryptanalysis with a SAT/Satisfiability modulo theories (SMT) solver-based automatic search method on these variants. SAECHAM is the best variant selected from those we analyzed. Table I presents the best known differential and linear distinguishers for CHAM64 and SAECHAM. The results show that SAECHAM offers similar resistance to linear attacks and provides better resistance to differential attacks compared to CHAM64.

For SAECHAM, we experimentally verify that it is indeed more efficient than CHAM64. Through software implementations on 8-bit AVR, 16-bit MSP430, 32-bit ARM Cortex-M3 and Cortex-M4 platforms, we demonstrate that SAECHAM is more efficient than CHAM64 in terms of encryption speed. In addition, we compare the balanced implementations of various ARX ciphers, including CHAM64 and SAECHAM. A balanced implementation refers to one that takes into account not only encryption speed but also code size and RAM usage in 8-bit AVR and 16-bit MSP430 environments. While performance on embedded devices is important for lightweight

TABLE I
COMPARISON OF OPTIMAL DISTINGUISHERS

Cipher	Cryptanalysis	Round	Pr / Cor	Ref
CHAM64	Differential	39	2^{-63}	[2]
	Linear	36	2^{-31}	[4]
	Linear	37	2^{-32}	[4]
SAECHAM	Differential	38	2^{-61}	Sect. IV
	Differential	39	2^{-65}	
	Linear	36	2^{-31}	
	Linear	37	2^{-32}	

Pr: Differential probability, Cor: Linear correlation

TABLE II
SUMMARY OF SPEED COMPARISON

Environment	SAECHAM	CHAM64
8-bit AVR	137.0 cpb	148.1 cpb [11]
16-bit MSP	99.8 cpb	114.6 cpb
ARM Cortex-M3	84.0 cpb	89.6 cpb
ARM Cortex-M4	70.8 cpb	82.4 cpb
NEON	5.865 cpb	7.554 cpb
AVX2	1.885 cpb	2.071 cpb
AVX512	1.503 cpb	1.631 cpb

block ciphers, encryption speed on high-performance devices, such as servers is also important because data collected from embedded devices can be used for cloud computing and big data processing. Therefore, we also compare the performance of parallel encryption using single instruction multiple data (SIMD) instructions AVX and NEON. The implementation results of SAECHAM on various devices can be found in the following repository <https://github.com/dampers/SAECHAM-implementations>.

Table II summarizes the comparison of speed on variable instruction sets. The cpb refers to clock cycles per byte. On low-power devices, such as the 8-bit AVR and 16-bit MSP430, SAECHAM demonstrates approximately 8.1% and 14.7% faster speeds compared to CHAM64, respectively. On 32-bit ARM Cortex-M3 and Cortex-M4, SAECHAM achieves 6.6% and 16.3% faster speeds, respectively. In SIMD instruction environments, it provides a speed improvement over CHAM64 of 28.7% on NEON, 9.8% on AVX2, and 8.5% on AVX512.

D. Organization

The outline of the article is as follows. In Section II, we provide notations, description of CHAM64, brief introduction to automatic search with SMT solver and how to calculate differential probability and linear correlation for ARX ciphers. Section III introduces CHAM-like structures and how to simplify CHAM-like structures by rotation amount. In Section IV, we present the linear correlation and differential probability of CHAM64 variants and define the best CHAM64 variant SAECHAM. In Section V we provide comparisons of the software implementations of CHAM64 and SAECHAM. Finally, we conclude this article in Section VI.

II. PRELIMINARIES

A. Notations

In this article, we use the following notations.

- $X_i[j]$ j -th word of i -round input state.
- $x||y$ Concatenation of x and y .
- $x \boxplus y$ Addition of w -bit words x and y modulo 2^w .
- $x \oplus y$ XOR of x and y .
- $\text{ROL}_t(x)$ Rotation of w -bit word x to the left by t bits.
- $x \ll t$ Shift of x to the left by t bits.
- $x \gg t$ Shift of x to the right by t bits.
- $\neg x$ The negation of x .
- $\text{wt}(x)$ Hamming weight of x .
- $x \cdot y$ Inner product of n -bit vectors x and y .
- $x \preceq y$ For every bit index i from 0 to $w - 1$, each x_i is less than or equal to y_i .

B. Description of CHAM64

CHAM is a family of lightweight block ciphers composed of three versions with a 4-branch general Feistel network (GFN) type-1 structure. It is denoted by CHAM- n/k , n is the block size and k is the key size. w is the word size, so that when n is 64, w is 16, and when n is 128, w is 32. The three versions within the CHAM family are CHAM-64/128, CHAM-128/128, and CHAM-128/256. In [2], A revised version of CHAM with the numbers of rounds increased is proposed in order to ensure a sufficient security margin. The numbers of rounds, denoted by r , of CHAM-64/128, CHAM-128/128, and CHAM-128/256 are raised from 80, 80, and 96 to 88, 112, and 120, respectively. In this article, we focus on the revised version of CHAM-64/128, which is referred to as CHAM64.

The encryption process of CHAM64 makes a plaintext P , which consists of four 16-bit words ($X_0[0]||X_0[1]||X_0[2]||X_0[3]$), into a ciphertext $C(= X_r[0]||X_r[1]||X_r[2]||X_r[3])$ by applying a round function r times. Fig. 1 depicts the 2-round encryption of CHAM family. For the i th round, where $0 \leq i < r$, the round function takes the rotation amounts depending on the least significant bit (LSB) of i . If i is even, the round function is defined as

$$X_{i+1}[3] = \text{ROL}_8((X_i[0] \oplus i) \boxplus (\text{ROL}_1(X_i[1]) \oplus \text{RK}_{(i \bmod 2k/w)}))$$

and if i is odd, the round function is

$$X_{i+1}[3] = \text{ROL}_1((X_i[0] \oplus i) \boxplus (\text{ROL}_8(X_i[1]) \oplus \text{RK}_{(i \bmod 2k/w)}))$$

where $\text{RK}_{(i \bmod 2k/w)}$ is the round key. Additionally, for $0 \leq j < 3$, the remaining words are updated as follows:

$$X_{i+1}[j] = X_i[j + 1].$$

The key schedule of CHAM family takes the k -bit master key K , where $K = K_0||K_1||\dots||K_{k/w-1}$ and generates the $2k/w$ w -bit round keys as follows:

$$\text{RK}_i = K_i \oplus \text{ROL}_1(K_i) \oplus \text{ROL}_8(K_i)$$

$$\text{RK}_{(i+k/w) \oplus 1} = K_i \oplus \text{ROL}_1(K_i) \oplus \text{ROL}_{11}(K_i)$$

where $0 \leq i < k/w$.

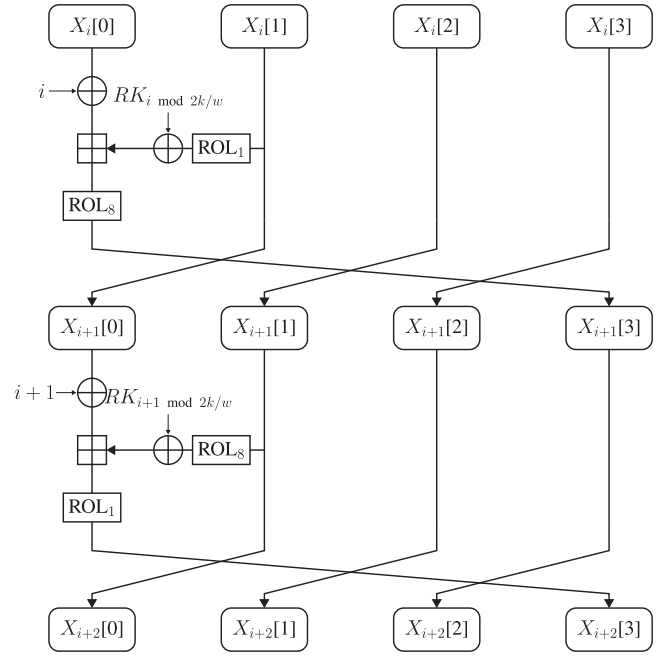


Fig. 1. 2-round encryption function of CHAM family.

C. Differential Probability for ARX Cipher

Differential cryptanalysis, proposed by Biham and Shamir [15], is a chosen-plaintext attack to analyze block ciphers by examining how differences in the input propagate to differences in the output. If a block cipher, for a given input difference $\Delta_{\text{in}} (= P \oplus P')$, produces a specific output difference $\Delta_{\text{out}} (= C \oplus C')$ with a significantly higher probability than random processes, this can be exploited to construct distinguishers or even recover parts of the encryption key.

To evaluate the security of block ciphers against differential cryptanalysis, Lai et al. [16] introduced the concept of Markov ciphers. In Markov ciphers, the probability of a differential characteristic is calculated by multiplying the transition probabilities of differences in each round, and the overall probability of a differential is the product of the probabilities of all the related differential characteristics.

For a vector Boolean function $f: F_2^n \rightarrow F_2^n$, the differential probability $\text{dp}(\Delta_{\text{in}} \xrightarrow{f} \Delta_{\text{out}})$ of a differential transition $\Delta_{\text{in}} \xrightarrow{f} \Delta_{\text{out}}$, where Δ_{in} is an n -bit input XOR-difference and Δ_{out} is an n -bit output XOR-difference, is defined as follows:

$$\text{dp}\left(\Delta_{\text{in}} \xrightarrow{f} \Delta_{\text{out}}\right) = \frac{|\{P \in \mathbf{P} : f(P) \oplus f(P \oplus \Delta_{\text{in}}) = \Delta_{\text{out}}\}|}{2^n}$$

where $\mathbf{P}$ represents the set of all possible plaintexts.

In ARX block ciphers, the only nonlinear operation is addition modulo 2^w , which causes differences to propagate. Thus, we focus on calculating the differential probability of the addition.

Let $\text{xdp}^+(\alpha, \beta \rightarrow \gamma)$ XOR-differential probability of addition modulo 2^w , with input differences α and β and output difference γ . In [17], Lipmaa and Moriai prove that the differential $(\alpha, \beta \rightarrow \gamma)$ is valid if and only if

$$\text{eq}(\alpha \ll 1, \beta \ll 1, \gamma \ll 1) \wedge (\alpha \oplus \beta \oplus \gamma \oplus (\beta \ll 1)) = 0$$

where

$$\text{eq}(x, y, z) := (\neg x \oplus y) \wedge (\neg x \oplus z).$$

For every valid differential $(\alpha, \beta \rightarrow \gamma)$, we define $\text{weight}(\alpha, \beta \rightarrow \gamma)$ as

$$\text{weight}(\alpha, \beta \rightarrow \gamma) := -\log_2(\text{xdp}^+(\alpha, \beta \rightarrow \gamma)).$$

The weight of a valid differential can be calculated as

$$\text{weight}(\alpha, \beta \rightarrow \gamma) := -\text{wt}^*(\neg \text{eq}(x, y, z))$$

where $\text{wt}^*(x)$ is the hamming weight of x of which the most significant bit (MSB) is excepted.

D. Linear Correlation for ARX Cipher

Linear cryptanalysis, introduced by Matsui in 1993 [18], is a highly effective cryptanalytic technique for block ciphers. This known-plaintext attack involves examining the probabilistic connections between plaintext, ciphertext, and the key through linear approximations. These approximations reveal statistical biases that highlight correlations between specific bits of plaintext, ciphertext, and the key. By analyzing multiple plaintext-ciphertext pairs, an attacker can use these linear approximations to deduce parts of the key.

For a vector boolean function $f: F_2^n \rightarrow F_2^n$, the correlation $\text{cor}(\Gamma_{\text{in}} \xrightarrow{f} \Gamma_{\text{out}})$ of a linear approximation $\Gamma_{\text{in}} \xrightarrow{f} \Gamma_{\text{out}}$, using an n -bit input mask Γ_{in} and an n -bit output mask Γ_{out} , is defined as follows:

$$\text{cor}\left(\Gamma_{\text{in}} \xrightarrow{f} \Gamma_{\text{out}}\right) = \frac{|\{x: \Gamma_{\text{in}} \cdot x = \Gamma_{\text{out}} \cdot f(x)\}|}{2^{n-1}} - 1.$$

When the function $g = f_{r-1} \circ \dots \circ f_1 \circ f_0$ is composed of r round functions f_i , the Piling-up Lemma introduced by Matsui states that linear approximation $\Gamma_{\text{in}} \xrightarrow{f} \Gamma_{\text{out}}$ of each round f_i can be concatenated into a linear trail $(\Gamma_0, \Gamma_1, \dots, \Gamma_r)$ of the function g .

Lemma 1 (Piling-Up Lemma [18]): Let $(\Gamma_0, \Gamma_1, \dots, \Gamma_r)$ be a linear trail of an r -round encryption function. Then the correlation of the linear trail can be calculated as

$$\text{cor}\left(\Gamma_0 \xrightarrow{f} \Gamma_r\right) = \prod_{i=0}^{r-1} \text{cor}\left(\Gamma_i \xrightarrow{f} \Gamma_{i+1}\right).$$

In [19], Schulte-Geers proved an explicit result for computing the correlation of linear approximations in addition modulo 2^w . Let $\text{cor}(m_1, m_2 \rightarrow m_3)$ denote the correlation of linear approximation of addition modulo 2^w , with input mask m_1 and m_2 and output mask m_3 . Here, z is a w -bit vector satisfying $z_{w-1} = 0$ and $z \oplus (z \gg 1) \oplus ((m_1 \oplus m_2 \oplus m_3) \gg 1) = 0$. Let us define $u = m_1 \oplus m_3$ and $v = m_2 \oplus m_3$. Then the correlation of the linear approximation is given by

$$\text{cor}(m_1, m_2 \rightarrow m_3) = 1_{\{u \preceq z\}} 1_{\{v \preceq z\}} (-1)^{u \cdot v} 2^{-\text{wt}(z)}.$$

The characteristic function $1_{\{x \preceq y\}}$ is defined as

$$1_{\{x \preceq y\}} = \begin{cases} 1, & \text{if } x \preceq y, \\ 0, & \text{otherwise.} \end{cases}$$

E. Automatic Search With SMT Solver

The Boolean Satisfiability Problem (SAT) involves assessing whether a given boolean formula can be satisfied by assignment of logical values to its variables. Specifically, it seeks to determine if there is a consistent way to assign TRUE or FALSE to each variable such that the entire formula evaluates to TRUE. When such an assignment exists, the formula is classified as satisfiable. On the other hand, if no such assignment can achieve a TRUE result, the formula is classified as unsatisfiable, meaning it evaluates to FALSE for all possible variable assignments. SMT problems can be seen as a generalised form of SAT problems. Whereas SAT problems determine whether a propositional formula can be satisfied by assigning true or false values to its variables, SMT problems determine whether a formula involving more expressive constructs can be satisfied under certain theoretical constraints. These theories can range from simple arithmetic and bit-vector theory to complex structures, such as arrays and algebraic data types. Some SMT solvers support bit-vector theory, which is suitable for modeling ARX ciphers. Bit-vector theory includes fixed-size bit-vector variables, functions for extracting individual bits from the vector, modular addition, shift, rotation, bitwise logical operators, and comparison functions.

Some SAT/SMT solvers are suitable for cryptanalysis. Cryptominisat [20] is a SAT solver that supports XOR operations, which are common in cryptographic problems. It also supports multithreading, allowing it to run efficiently across multiple CPU cores to solve large and complex SAT instances faster, making it especially useful for cryptographic analysis and optimization tasks.

STP [21] is a SMT solver that encodes constraints using the CVC and SMTLIB2 languages, and then calls the SAT solver to determine the satisfiability of the model. Mouha and Preneel [22] built a framework for finding the optimal differential property of ARX ciphers using the SMT solver STP and applied it to salsa20. There have been subsequent applications of STP to cryptanalysis, such as [23], [24], [25].

III. CHAM-LIKE STRUCTURE

The round function of CHAM is a 4-branch generalized Feistel network type-1 structure and has a very simple computational structure. In [14], CHAM-like structures were defined by changing only the order of operations in the round function of CHAM. In this section, we analyze the number of instructions of the CHAM-like structures according to the amount of rotation on AVR and MSP430.

The round function of CHAM is performed in the order of XOR round constant - addition - rotation (XAR). By changing the order of this operation structure, six different permutations of the structure can be constructed. Fig. 2 shows the six CHAM-like structures by changing the order of operation. In this study, to identify the operation structure, we list the leading characters of each operation and denote it as: 1) XAR; 2) XRA; 3) RAX; 4) RXA; 5) AXR; and 6) ARX.

The rotation amounts in the 2-round encryption of a CHAM-like structure are represented as a 4-tuple (a, b, c, d) . In Fig. 2,

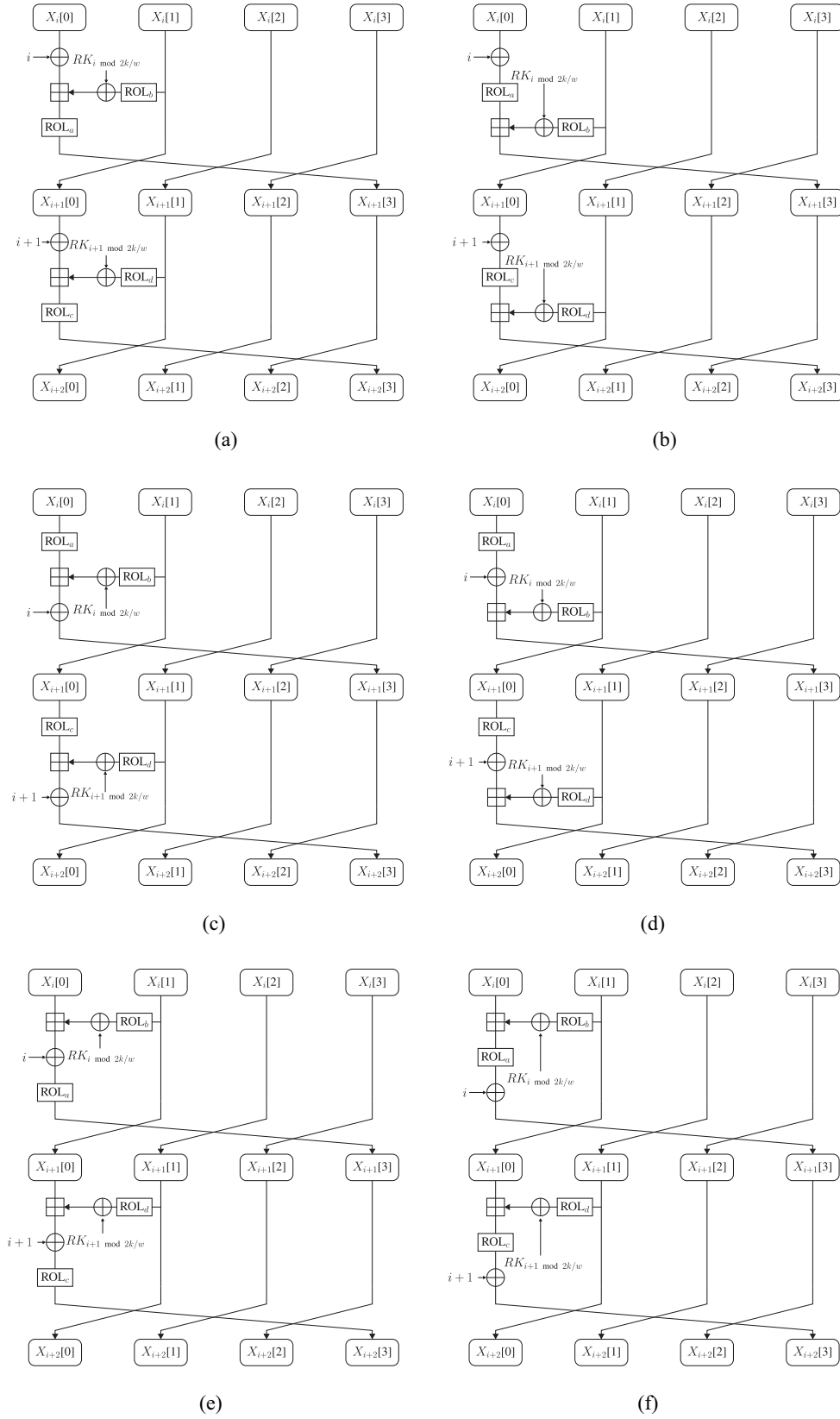


Fig. 2. 2-round encryption of CHAM-like structures. (a) CHAM-XAR. (b) CHAM-XRA. (c) CHAM-RAX. (d) CHAM-RXA. (e) CHAM-AXR. (f) CHAM-ARX.

a denotes the rotation size applied to the $X_i[0]$ in even rounds (the i is even.) and b represents the rotation size applied to the $X_i[1]$ before the addition in even rounds. Similarly, c denotes

the rotation size applied to the $X_{i+1}[0]$ in odd rounds, and d represents the rotation size applied to the $X_{i+1}[1]$ before the addition in odd rounds.

TABLE III
NUMBER OF INSTRUCTION OF 16-BIT ROTATIONS

MCU \ ROL	1	7	8	9	15
8-bit AVR	3	4	0	3	4
16-bit MSP430	2	3	1	3	2

TABLE IV
ASSEMBLY CODES FOR 16-BIT ROTATIONS IN 8-BIT
AVR AND 16-BIT MSP430

MCU	8-bit AVR		16-bit MSP430
Register	hi, lo		a
ROL ₁	1. LSL	lo	1. RLA a
	2. ROL	hi	2. ADC a
	3. ADC	lo, R1	
ROL ₇	1. BST	hi, 0	1. SWPB a
	2. ROR	lo	2. BIT #1, a
	3. ROR	hi	3. RRC a
	4. BLD	lo, 7	
ROL ₈			1. SWPB a
ROL ₉	1. LSL	hi	1. SWPB a
	2. ROL	lo	2. RLA a
	3. ADC	hi, R1	3. ADC a
ROL ₁₅	1. BST	lo, 0	2. BIT #1, a
	2. ROR	hi	3. RRC a
	3. ROR	lo	
	4. BLD	hi, 7	

In round function of CHAM, the rotation amounts were selected with careful consideration of security and efficiency. In particular, we accounted for the variation in implementation costs associated with different rotation amounts on 8-bit AVR and 16-bit MSP430 architectures. When implementing 16-bit rotation ROL_t on 8-bit AVR and 16-bit MSP430, ROL_t can be implemented by iterating ROL₁ operation t times. In the case of ROL₈, on 8-bit AVR, it is equivalent to a register swap, and on 16-bit MSP430, it can be implemented using the swpb instruction. Consequently, the rotation amounts in CHAM were chosen to be as close as possible to 0 or 8. Reflecting this design principle of CHAM, the search space for rotation amounts in CHAM64 variants was reduced to 1, 7, 8, 9, and 15. Table III presents the number of instructions required for each rotation amount on AVR and MSP430.

The number of instructions for ROL₇ corresponds to the sum of those required for ROL₈ and ROL₁₅, while the number of instructions for ROL₉ equals the sum of those for ROL₈ and ROL₁. On 8-bit AVR, ROL₈ can be implemented via a word-level swap, but it can also be realized without additional instructions through register scheduling. Detailed assembly codes can be found in Table IV.

Some CHAM-like structures can cancel the rotation operation if certain conditions on the position of the rotation operation and the amount of rotation are satisfied. Among CHAM-like structures, CHAM-XAR and CHAM-AXR, of which the third operation is rotation, can cancel ROL_d if $a + d = w$ after the second round. Fig. 3 shows the initial 8-round encryption function of CHAM64 that is CHAM-XAR

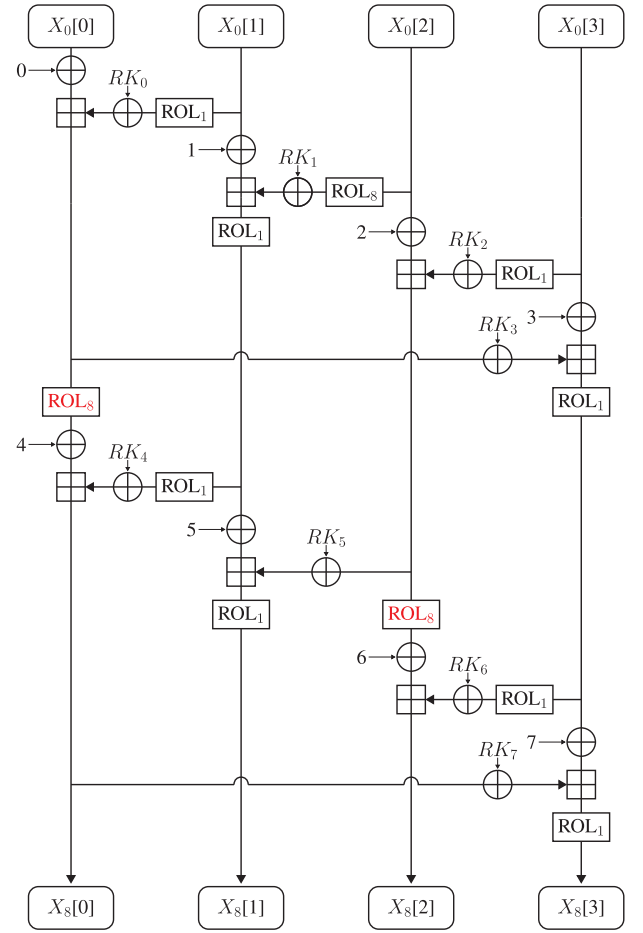


Fig. 3. Initial 8-round encryption of CHAM64 without word swap.

structure and has $a = 8$ and $d = 8$ without word swap. Before performing the modular addition in the third round, the XOR operation is performed on $X[0]$ with RK_3 after the ROL₈ is performed on $X[0]$. In this case, we have performed two successive ROL₈ on $X[0]$, which is equivalent to not performing the ROL operation because the word size is 16. Therefore, if we remove the ROL₈ operations on $X[0]$ in the 0th and third rounds, and perform the ROL₈ operation at the beginning of the fourth round, we can remove a ROL₈ operation. For the same reason that $a + d = w$, we can remove ROL_b after the third round if $b + c = w$.

Similarly, for CHAM-RXA and CHAM-RAX structures, where the rotation is the first operation, ROL_d can be removed starting from the 1st round when $a = d$, and ROL_b can be removed when $b = c$. Fig. 4 illustrates the initial 8-round encryption function of the CHAM64-RXA structure without word swap when $a = d$. The separately calculated ROL_d in the 1st round and ROL_a in the second round can be combined into a single operation ROL_a. And similarly, ROL_b can be combined when $b = c$.

In this article, we do not consider changing the order of the round key XOR and the rotation performed on $X_i[1]$ branch. Swapping the order of these operations does not affect security in terms of differential and linear analysis, but does not improve efficiency either. If these operations are swapped,

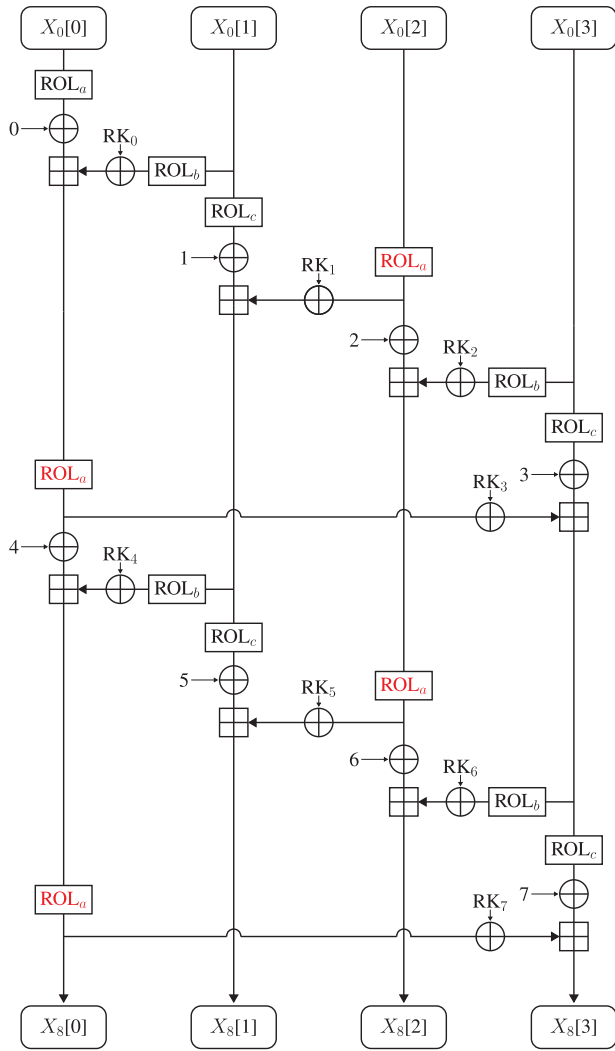


Fig. 4. Initial 8-round encryption of CHAM64-RXA structure without word swap when $a = d$.

the barrel shifter provided in the ARM environment cannot be used for this part, and as shown in the structural analysis, the rotation cannot be eliminated.

CHAM64 Variants: Based on the above structural analysis, CHAM64 variants can be defined as follows. In the notation $(a, b, c, d)_{(C_1, C_2)}$, (a, b, c, d) represents the rotation amount permutation, while C_1 and C_2 denote the number of instructions for rotations in the 2-round encryption function on 8-bit AVR and 16-bit MSP430 platforms, respectively. For example, CHAM64 can be redefined as CHAM64-XAR $(8, 1, 1, 8)_{(6, 5)}$. As mentioned earlier, since CHAM64 has the CHAM-XAR structure and satisfies $a + d = w$, one ROL_8 can be removed in 2-round encryption. Therefore on 8-bit AVR, C_1 is $0+3+3 = 6$, and on 16-bit MSP430, C_2 is $1+2+2 = 5$.

IV. SECURITY ANALYSIS ON CHAM VARIANTS

The security, as well as efficiency, varies depending on the rotation amount. In this section, we analyze the resistance to linear and differential cryptanalysis by searching the optimal linear and differential trails of CHAM64 variants that have

rotation amounts requiring the same or fewer number of instructions than CHAM64 on AVR and MSP430. Lastly, we identify the best CHAM64 variant and define it as SAECHAM.

The six CHAM-like structures can be classified into two equivalence classes for the differential and linear analysis: CHAM-AR and CHAM-RA. CHAM-XAR, CHAM-AXR, and CHAM-ARX belong to the CHAM-AR class, while the remaining three structures, CHAM-XRA, CHAM-RXA, and CHAM-RAX, belong to the CHAM-RA class. Structures within the same class exhibit identical linear and differential properties. The rotation-eliminable pairs (CHAM-AXR, CHAM-XAR) and (CHAM-RXA, CHAM-RAX) belong to the CHAM-AR and CHAM-RA, respectively. Consequently, we focus on searching for the linear and differential characteristics of CHAM64 variants that require the same or fewer instructions than CHAM64.

A. Linear Characteristics of CHAM64 Variants

In this section, we aim to find CHAM64 variants with linear attack resistance equal to or greater than that of CHAM64, using a framework-based on an SMT solver. Specifically, we search for a CHAM64 variant whose correlation is equal to or lower than that of CHAM64, on 36 rounds, which is the maximum number of rounds where the correlation is less than 2^{-32} . Table V shows the linear correlations of 36-round CHAM64 variants for rotation amount permutations that require the same or fewer instructions than CHAM64, within the CHAM-AR and CHAM-RA classes. In the notation $(a, b, c, d)_{(C_1, C_2)}$, (a, b, c, d) represents the rotation amount permutation, while C_1 and C_2 denote the number of instructions for rotations in the 2-round encryption function on 8-bit AVR and 16-bit MSP430 platforms, respectively.

The longest linear trail of CHAM64 with correlation bigger than 2^{-32} is 36-round linear trail [4] with correlation 2^{-31} so far. These findings align with our results for the CHAM-AR configuration CHAM64(CHAM-AR $(8, 1, 1, 8)_{(6, 5)}$) as presented in Table V. We observe that among the 36-round CHAM64 variants in the CHAM-RA class, the configurations with the same correlation value as CHAM64 are $(8, 8, 7, 8)_{(4, 5)}$ and $(7, 8, 8, 8)_{(4, 5)}$, while those with a lower correlation than CHAM64 are $(1, 1, 8, 1)_{(6, 5)}$ and $(8, 1, 1, 1)_{(6, 5)}$. Both $(8, 8, 7, 8)_{(4, 5)}$ and $(7, 8, 8, 8)_{(4, 5)}$ achieve a correlation of 2^{-32} for the 37-round optimal trail, making them the longest linear trails with a correlation greater than 2^{-32} for the 36-round case.

In the CHAM-AR class, aside from CHAM64(CHAM-AR $(8, 1, 1, 8)_{(6, 5)}$) itself, the only 36-round CHAM64 variant with the same correlation value as CHAM64 is $(1, 8, 8, 1)_{(6, 5)}$, and none exhibit a lower correlation. This means that within the CHAM-AR class, the designers selected rotation amounts that provide an optimal balance between security and cost.

B. Differential Characteristics of CHAM64 Variants

In this section, we aim to find CHAM variants with differential attack resistance equal to or greater than that

TABLE V
LINEAR CORRELATIONS OF 36-ROUND CHAM64 VARIANTS

CHAM-RA	Correlation	CHAM-AR	Correlation
$(1, 8, 8, 1)_{(3,3)}$	2^{-26}	$(1, 8, 8, 1)_{(3,3)}$	2^{-24}
$(8, 1, 1, 8)_{(3,3)}$	2^{-26}	$(8, 1, 1, 8)_{(3,3)}$	2^{-23}
$(8, 8, 1, 8)_{(3,4)}$	2^{-24}	$(8, 8, 1, 8)_{(3,4)}$	2^{-30}
$(8, 8, 8, 1)_{(3,4)}$	2^{-18}	$(8, 8, 8, 1)_{(3,4)}$	2^{-24}
$(8, 9, 9, 8)_{(3,4)}$	2^{-27}	$(8, 9, 9, 8)_{(3,4)}$	2^{-18}
$(9, 8, 8, 9)_{(3,4)}$	2^{-27}	$(9, 8, 8, 9)_{(3,4)}$	2^{-24}
$(1, 8, 8, 8)_{(3,4)}$	2^{-24}	$(1, 8, 8, 8)_{(3,4)}$	2^{-30}
$(8, 1, 8, 8)_{(3,4)}$	2^{-18}	$(8, 1, 8, 8)_{(3,4)}$	2^{-18}
$(8, 8, 9, 8)_{(3,5)}$	2^{-26}	$(8, 8, 9, 8)_{(3,5)}$	2^{-27}
$(8, 8, 8, 9)_{(3,5)}$	2^{-18}	$(8, 8, 8, 9)_{(3,5)}$	2^{-18}
$(9, 8, 8, 8)_{(3,5)}$	2^{-26}	$(9, 8, 8, 8)_{(3,5)}$	2^{-28}
$(8, 9, 8, 8)_{(3,5)}$	2^{-18}	$(8, 9, 8, 8)_{(3,5)}$	2^{-18}
$(8, 15, 15, 8)_{(4,3)}$	2^{-24}	$(8, 15, 15, 8)_{(4,3)}$	2^{-26}
$(15, 8, 8, 15)_{(4,3)}$	2^{-23}	$(15, 8, 8, 15)_{(4,3)}$	2^{-26}
$(7, 8, 8, 7)_{(4,4)}$	2^{-24}	$(7, 8, 8, 7)_{(4,4)}$	2^{-24}
$(15, 8, 8, 8)_{(4,4)}$	2^{-30}	$(15, 8, 8, 8)_{(4,4)}$	2^{-19}
$(8, 8, 8, 15)_{(4,4)}$	2^{-19}	$(8, 8, 8, 15)_{(4,4)}$	2^{-24}
$(8, 8, 15, 8)_{(4,4)}$	2^{-30}	$(8, 8, 15, 8)_{(4,4)}$	2^{-28}
$(8, 7, 7, 8)_{(4,4)}$	2^{-24}	$(8, 7, 7, 8)_{(4,4)}$	2^{-19}
$(8, 15, 8, 8)_{(4,4)}$	2^{-19}	$(8, 15, 8, 8)_{(4,4)}$	2^{-28}
$(8, 8, 7, 8)_{(4,5)}$	2^{-31}	$(8, 8, 7, 8)_{(4,5)}$	2^{-28}
$(8, 7, 8, 8)_{(4,5)}$	2^{-19}	$(8, 7, 8, 8)_{(4,5)}$	2^{-19}
$(8, 8, 8, 7)_{(4,5)}$	2^{-19}	$(8, 8, 8, 7)_{(4,5)}$	2^{-19}
$(7, 8, 8, 8)_{(4,5)}$	2^{-31}	$(7, 8, 8, 8)_{(4,5)}$	2^{-27}
$(9, 1, 1, 9)_{(6,5)}$	2^{-19}	$(9, 1, 1, 9)_{(6,5)}$	2^{-16}
$(1, 8, 1, 1)_{(6,5)}$	2^{-18}	$(1, 8, 8, 1)_{(6,5)}$	2^{-31}
$(1, 1, 1, 8)_{(6,5)}$	2^{-18}	CHAM64 [4]	2^{-31}
$(1, 1, 8, 1)_{(6,5)}$	2^{-33}	$(1, 1, 8, 1)_{(6,5)}$	2^{-23}
$(1, 9, 9, 1)_{(6,5)}$	2^{-19}	$(1, 9, 9, 1)_{(6,5)}$	2^{-17}
$(8, 1, 1, 1)_{(6,5)}$	2^{-33}	$(8, 1, 1, 1)_{(6,5)}$	2^{-30}
		$(1, 1, 8, 1)_{(6,5)}$	2^{-30}
		$(1, 9, 9, 1)_{(6,5)}$	2^{-24}
		$(8, 1, 1, 1)_{(6,5)}$	2^{-18}

of CHAM64, using a framework based on an SMT solver. We focus our search on permutations that are as good as or better than CHAM64 in terms of linear cryptanalysis, as identified in Section IV-A. Specifically, in the CHAM-RA class, we search the differential trails for the permutations $(8, 8, 7, 8)_{(4,5)}$, $(7, 8, 8, 8)_{(4,5)}$, $(1, 1, 8, 1)_{(6,5)}$, and $(8, 1, 1, 1)_{(6,5)}$, while in the CHAM-AR class, we focus on the permutation $(1, 8, 8, 1)_{(6,5)}$.

In [2], a new differential characteristic for 39-round CHAM64 was presented using an SAT-based search method, and in [3], a new differential characteristic for 39-round CHAM64 was presented using a combinational DDT-based branch and bound search algorithm. These results currently represent the longest known differential trails for CHAM64. Therefore, in this section, we search for 39-round differential

trails for CHAM64 variants and identify the best CHAM64 variant.

Table VI shows the maximum probability of 39-round differential trails for CHAM64 variants. CHAM64-RA $(8, 1, 1, 1)_{(6,5)}$, like CHAM64, has a maximum probability of 2^{-63} for the 39-round differential trail and a maximum differential probability of 2^{-65} for the 40-round trail, making the 39-round trail the longest differential trail with a probability greater than 2^{-64} . The CHAM64 variant that outperforms CHAM64 in both linear correlation and differential probability is CHAM64-RA $(1, 1, 8, 1)_{(6,5)}$. However, considering the number of operations, CHAM64-RA $(1, 1, 8, 1)_{(6,5)}$ is expected to have a similar encryption speed to CHAM64. Among the CHAM64 variants that have fewer operations than CHAM64 and show better results in terms of differential

TABLE VI
DIFFERENTIAL PROBABILITIES OF 39-ROUND CHAM64 VARIANTS

CHAM64 variant	Differential probability
CHAM64-RA (8, 8, 7, 8) _(4,5)	2^{-65}
CHAM64-RA (7, 8, 8, 8) _(4,5)	2^{-64}
CHAM64-RA (1, 1, 8, 1) _(6,5)	2^{-64}
CHAM64-RA (8, 1, 1, 1) _(6,5)	2^{-63}
CHAM64-AR (1, 8, 8, 1) _(6,5)	2^{-62}
CHAM64 [2]	2^{-63}

and linear cryptanalysis, CHAM64-RA (8, 8, 7, 8)_(4,5) and CHAM64-RA (7, 8, 8, 8)_(4,5) stand out.

Therefore, we define CHAM64-RXA (8, 8, 7, 8)_(4,5) as SAECHAM which shows better resistance to differential cryptanalysis.

C. Description of SAECHAM

SAECHAM is an ARX-based lightweight block cipher that supports 64-bit block size and 128-bit key size, and has 88 rounds. Fig. 5 illustrates the 2-round encryption function of SAECHAM. The encryption process for the even round is defined as

$$X_{i+1}[3] = (\text{ROL}_8(X_i[0]) \oplus i) \boxplus (\text{ROL}_8(X_i[1]) \oplus \text{RK}_{(i \bmod 2k/w)})$$

while for the odd round, it is defined as:

$$X_{i+1}[3] = (\text{ROL}_8(X_i[0]) \oplus i) \boxplus (\text{ROL}_7(X_i[1]) \oplus \text{RK}_{(i \bmod 2k/w)})$$

where $\text{RK}_{(i \bmod 2k/w)}$ denotes the round key. Similar to CHAM64, the remaining words for $0 \leq j < 3$ are updated as follows:

$$X_{i+1}[j] = X_i[j + 1].$$

The key schedule of SAECHAM is the same as that of CHAM64, and a test vector is as follows:

master key	0x0100 0x0302 0x0504 0x0706
	0x0908 0x0B0A 0x0D0C 0x0F0E
plaintext	0x1100 0x3322 0x5544 0x7766
ciphertext	0xFE47 0x5393 0xE6BA 0x01F1

SAECHAM adopts the CHAM-RXA structure and satisfies the condition $a = d$, allowing it to be simplified when implemented as shown in Fig. 6 and mentioned in Section III.

The maximum linear correlations and maximum differential probabilities for each round of SAECHAM are shown in Tables VII and VIII. The optimal linear trails for the 36-round SAECHAM and differential trails for the 38-round SAECHAM can be found in Appendix B.

V. SOFTWARE IMPLEMENTATIONS

This section presents an evaluation of performance of SAECHAM across various devices under a fixed-key scenario [26]. First, we compare SAECHAM and CHAM64 implementations on 8-bit AVR and 16-bit MSP430. The

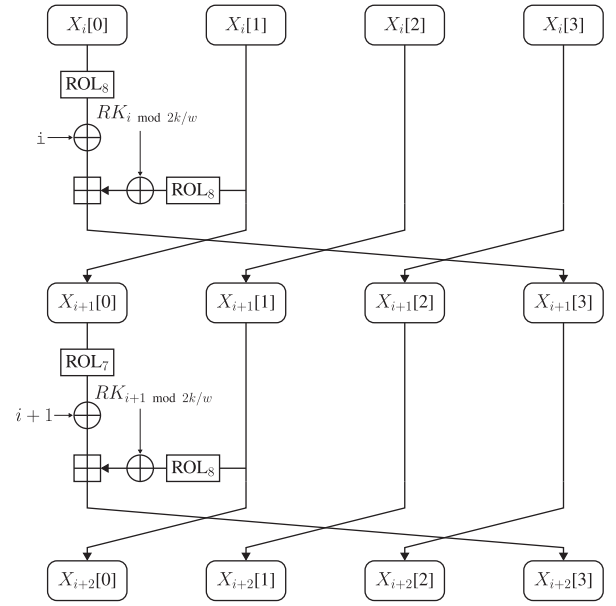


Fig. 5. 2-round encryption function of SAECHAM.

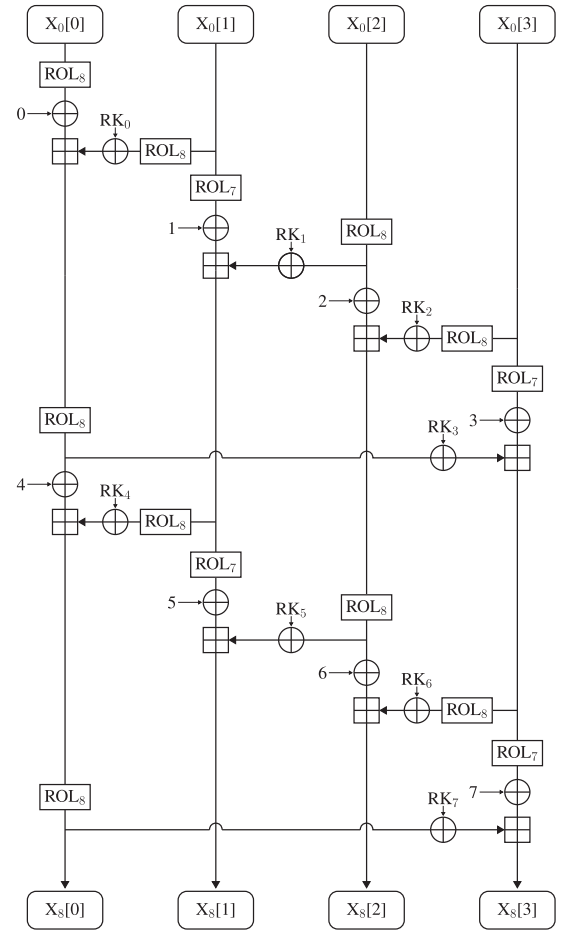


Fig. 6. Initial 8-round encryption of SAECHAM without word swap.

efficient implementation on 8-bit AVR and 16-bit MSP430 is one of the design principles of SAECHAM and the CHAM family.

TABLE VII
MAXIMUM LINEAR CORRELATIONS FOR EACH ROUND OF **SAECHAM**

Round	Correlation	Round	Correlation
1	1	20	2^{-11}
2	1	21	2^{-12}
3	1	22	2^{-13}
4	1	23	2^{-14}
5	1	24	2^{-16}
6	1	25	2^{-17}
7	2^{-1}	26	2^{-18}
8	2^{-1}	27	2^{-20}
9	2^{-2}	28	2^{-21}
10	2^{-2}	29	2^{-22}
11	2^{-3}	30	2^{-24}
12	2^{-4}	31	2^{-25}
13	2^{-5}	32	2^{-26}
14	2^{-6}	33	2^{-27}
15	2^{-6}	34	2^{-29}
16	2^{-7}	35	2^{-30}
17	2^{-7}	36	2^{-31}
18	2^{-8}	37	2^{-32}
19	2^{-10}	38	2^{-34}

TABLE VIII
MAXIMUM DIFFERENTIAL PROBABILITIES FOR
EACH ROUND OF **SAECHAM**

Round	Probability	Round	Probability
1	1	21	2^{-19}
2	1	22	2^{-20}
3	1	23	2^{-23}
4	1	24	2^{-25}
5	1	25	2^{-28}
6	2^{-1}	26	2^{-30}
7	2^{-2}	27	2^{-33}
8	2^{-3}	28	2^{-36}
9	2^{-3}	29	2^{-38}
10	2^{-4}	30	2^{-40}
11	2^{-4}	31	2^{-43}
12	2^{-5}	32	2^{-45}
13	2^{-7}	33	2^{-48}
14	2^{-8}	34	2^{-50}
15	2^{-10}	35	2^{-53}
16	2^{-11}	36	2^{-56}
17	2^{-12}	37	2^{-59}
18	2^{-13}	38	2^{-61}
19	2^{-16}	39	2^{-65}
20	2^{-18}	40	2^{-68}

The ARM Cortex-M3 and Cortex-M4 microcontrollers, both 32-bit MCUs, are widely used in IoT devices and real-time applications due to their balance of low power consumption

TABLE IX
PERFORMANCE COMPARISON OF FAST IMPLEMENTATIONS ON 8-BIT AVR
AND 16-BIT MSP (PRECOMPUTED ROUND KEYS)

Cipher	8-bit AVR	16-bit MSP
SAECHAM	137.0 cpb	99.8 cpb
CHAM64	148.1 cpb [11]	114.6 cpb

and high performance. Optimizing implementations for block cipher on these platforms is crucial to achieving secure and efficient data processing, especially in resource-constrained environments, such as IoT and industrial automation. In this context, we compare **SAECHAM** and **CHAM64** implementations on the ARM Cortex-M3 and Cortex-M4.

Beyond evaluating performance in constrained environments, it is crucial to consider how **SAECHAM** performs in high-performance settings, such as servers. Fast encryption is vital for maintaining operational efficiency and cost-effectiveness in data centers, cloud computing environments, and big data analysis platforms, where large volumes of data from IoT devices are processed. To assess whether **SAECHAM** maintains its advantages in such high-performance scenarios, we compare its parallel implementations using SIMD instructions in modern CPU architectures.

A. 8-Bit AVR and 16-Bit MSP430

In Section III, we analyzed the rotation costs associated with 8-bit AVR and 16-bit MSP430 platforms. This analysis allowed us to identify rotation amounts that are either equal to or less than the computational cost of **CHAM64**. Following the security analysis in Section IV, we determined an optimized rotation amount that incurs less overhead than **CHAM64**. In this context, **SAECHAM** is expected to outperform **CHAM64** on 8-bit AVR and 16-bit MSP430 platforms.

Since **CHAM** is designed with a focus on encryption speed, Table IX compares the fast implementations of **CHAM64** and **SAECHAM** on 8-bit AVR and 16-bit MSP430 platforms. The encryption speed is measured in terms of clock cycles per byte (cpb). The IDE for 8-bit AVR experiments is Microchip Studio ver 7.0, and the IDE for 16-bit MSP430 is Code Composer Studio ver 12.7.1. All encryption functions is implemented in assembly code. Details about the target platforms are provided in Appendix A. **SAECHAM** demonstrates approximately 8.1% faster performance than **CHAM64** on the 8-bit AVR and about 14.7% faster performance on the 16-bit MSP430. These results experimentally confirm that **SAECHAM** achieves superior encryption speed compared to **CHAM64** on both platforms, reinforcing its suitability for lightweight and resource-constrained environments.

In IoT environments, speed is important, but so are code size and RAM usage. In [26], the authors introduce the *rank* metric as an overall performance indicator. It is defined as

$$\text{rank} = (10^6/\text{cpb}) / (\text{ROM} + 2 \times \text{RAM})$$

where ROM and RAM refers the byte size of the memory of each type. Table X presents a comparative evaluation of block

TABLE X
PERFORMANCE COMPARISON WITH THE RANK METRIC [26] ON AVR AND MSP430 (PRECOMPUTED ROUND KEYS)

Cipher	8-bit AVR				16-bit MSP430				Ref.
	ROM	RAM	cpb	rank	ROM	RAM	cpb	rank	
SAECHAM	132	2	205	35.8	132	8	108	62.6	this work
CHAM64	202	3	188	28.0	156	8	128	45.1	[2]
SPECK-64/128	218	0	154	29.8	204	0	98	50.0	[26]
SPARX-64/128	248	2	223	17.7	166	14	136	37.9	[27]
SIMON-64/128	290	0	253	13.6	280	0	177	20.2	[26]
HIGHT	336	0	311	9.6	448	674	147	3.7	[26], [29]

ciphers using the *rank* metric, which balances speed, ROM, and RAM usage. The table includes results for **SAECHAM**, **CHAM64**, and other ARX-based lightweight ciphers, such as **SPECK-64/128** [9], **SIMON-64/128** [9], **SPARX-64/128** [27], and **HIGHT** [28] tested on both AVR and MSP430 platforms under a fixed-key scenario with precomputed round keys. All the listed ciphers have a 64-bit block size and a 128-bit key size. For each cipher, the ROM and RAM usage in bytes, clock cpb, and *rank* values are shown. Higher *rank* values indicate better overall performance, making **SAECHAM** the most efficient on both platforms.

B. Implementations on 32-Bit ARM Cortex-M3 and Cortex-M4

The ARM Cortex-M3 and Cortex-M4 are 32-bit microcontrollers widely used in IoT and embedded systems due to their excellent balance between performance and power efficiency. The Cortex-M4 includes DSP instructions consisting of MAC and limited SIMD instructions. We compare the implementations of **CHAM64** and **SAECHAM** on these MCUs. The IDE used for experiments in the ARM Cortex-M3 and Cortex-M4 is STM32CubeIDE ver 1.12.1, and all encryption functions are implemented in assembly code. Details about the target platforms are provided in Appendix A.

Since the ARM Cortex-M3 and Cortex-M4 are 32-bit MCUs, double-block parallel implementations are feasible for **CHAM64** and **SAECHAM**, which use 16-bit words. Moreover, the ARM Cortex-M4 supports simple SIMD instructions for 32-bit registers. For example, the `UADD16` instruction on the ARM Cortex-M4 performs parallel addition of two 32-bit registers by dividing each into upper and lower 16-bit halves and adding them independently.

The performance comparison in Table XI clearly demonstrates that **SAECHAM** is faster than **CHAM64** on both the ARM Cortex-M3 and ARM Cortex-M4, whether using single-block or double-block implementations. On the ARM Cortex-M3, **SAECHAM** achieves approximately 15% better performance in single-block encryption and 6.6% better performance in double-block encryption compared to **CHAM64**. Similarly, on the ARM Cortex-M4, **SAECHAM** performs about 33.2% faster in single-block encryption and 16.4% faster in double-block encryption. These results highlight the superior speed of **SAECHAM** over **CHAM64** in 32-bit ARMv7-M MCU environments.

TABLE XI
PERFORMANCE COMPARISON ON 32-BIT ARM CORTEX-M3 AND CORTEX-M4 (PRECOMPUTED ROUND KEYS)

Target MCU	Blocks	SAECHAM	CHAM64
ARM Cortex-M3	single	110.4 cpb	127.0 cpb
ARM Cortex-M3	double	84.0 cpb	89.6 cpb
ARM Cortex-M4	single	123.9 cpb	165.0 cpb
ARM Cortex-M4	double	70.8 cpb	82.4 cpb

SIMD instructions enable parallel processing by executing a single instruction on multiple data elements simultaneously. This capability is particularly useful for cryptographic algorithms, where efficiently processing large datasets is crucial. SIMD instruction sets, such as AVX2, AVX512, and NEON are widely supported in modern processors, allowing for optimized implementations of block ciphers. In the context of big data servers, where encrypted data from IoT devices must be rapidly processed, utilizing SIMD instructions for parallel block cipher implementations is essential.

Table XII compares the encryption speed of **CHAM64** and **SAECHAM** in AVX2, AVX512, and NEON environments. The AVX2 and AVX512 experiments were conducted on an Intel Xeon Gold 6230 @ 2.10 GHz processor, while the NEON experiments were performed on an ARM Cortex-A72 processor.

VI. CONCLUSION AND FUTURE WORK

A. Conclusion

In this article, we presented **SAECHAM**, a variant of the ARX block cipher **CHAM64** that improves both efficiency and security. In terms of efficiency, by changing the order of operations and the amount of rotation in **CHAM**, we identified rotation operations that could be removed, resulting in a reduction of operations and an improvement in encryption speed. This process resulted in 62 **CHAM64** variants with the same or fewer instructions than **CHAM64** on AVR and MSP430 environments. In terms of security, using automatic search methods, we identified five **CHAM64** variants that performed similarly or better than **CHAM64** in linear cryptanalysis, and among them, three variants that outperformed **CHAM64** in differential cryptanalysis. This led to the proposal of **SAECHAM**, a more secure and more efficient variant than

TABLE XII
PERFORMANCE COMPARISON PARALLEL IMPLEMENTATION USING SIMD INSTRUCTIONS (PRECOMPUTED ROUND KEYS)

Instruction Set	SAECHAM	CHAM64
AVX2	1.885 cpb (8.89 Gbps)	2.071 cpb (8.09 Gbps)
AVX512	1.503 cpb (11.12 Gbps)	1.631 cpb (10.21 Gbps)
NEON	5.865 cpb (406 Mbps)	7.554 cpb (315 Mbps)

TABLE XIII
OPTIMAL LINEAR TRAIL FOR 36-ROUND SAECHAM

Round	$\Gamma X_r[0]$	$\Gamma X_r[1]$	$\Gamma X_r[2]$	$\Gamma X_r[3]$	Correlation
0	0x0008	0x0608	0x0B03	0x1503	2^{-1}
1	0x0600	0x0B03	0x1503	0x0C00	2^{-1}
2	0x0903	0x1503	0x0C00	0x0003	2^{-2}
3	0x1800	0x0C00	0x0003	0x0309	2^{-1}
4	0x0000	0x0003	0x0309	0x000C	2^0
5	0x0003	0x0309	0x000C	0x0000	2^{-1}
6	0x0308	0x000C	0x0000	0x0100	2^{-2}
7	0x0200	0x0000	0x0100	0x0802	2^0
8	0x0100	0x0100	0x0802	0x0001	2^0
9	0x0000	0x0802	0x0001	0x0001	2^0
10	0x0802	0x0001	0x0001	0x0000	2^{-2}
11	0x0803	0x0001	0x0000	0x030C	2^{-2}
12	0x0600	0x0000	0x030C	0x0104	2^{-1}
13	0x0400	0x030C	0x0104	0x0004	2^{-1}
14	0x010C	0x0104	0x0004	0x0002	2^{-1}
15	0x0008	0x0004	0x0002	0x0C01	2^{-1}
16	0x0000	0x0002	0x0C01	0x0600	2^0
17	0x0002	0x0C01	0x0600	0x0000	2^{-6}
18	0x0400	0x0600	0x0000	0x010C	2^{-1}
19	0x0200	0x0000	0x010C	0x0006	2^0
20	0x0100	0x010C	0x0006	0x0001	2^0
21	0x000C	0x0006	0x0001	0x0001	2^{-1}
22	0x0002	0x0001	0x0001	0x0400	2^{-1}
23	0x0003	0x0001	0x0400	0x0300	2^{-1}
24	0x0000	0x0400	0x0300	0x0100	2^0
25	0x0400	0x0300	0x0100	0x0000	2^{-1}
26	0x0100	0x0100	0x0000	0x0003	2^0
27	0x0000	0x0000	0x0003	0x0001	2^0
28	0x0000	0x0003	0x0001	0x0000	2^0
29	0x0003	0x0001	0x0000	0x0000	2^{-1}
30	0x0000	0x0000	0x0000	0x0100	2^0
31	0x0000	0x0000	0x0100	0x0000	2^0
32	0x0000	0x0100	0x0000	0x0000	2^0
33	0x0100	0x0000	0x0000	0x0000	2^{-1}
34	0x0080	0x0000	0x0000	0xC000	2^{-1}
35	0x0080	0x0000	0xC000	0xC000	2^{-1}
36	0x0060	0xC000	0xC000	0x4000	-

CHAM64. This design aligns with the principles of CHAM, which emphasized encryption speed, especially for frequent use in block cipher modes of operation in ISO/IEC standards, rather than focusing on a balance between encryption and decryption speed.

B. Future Work

For CHAM128/*, the number of variants is 531, and its larger block size than 64-bit block makes thorough analysis challenging. Finding the optimal linear or differential trail is particularly difficult. For instance, an optimal linear trail for CHAM128/* has not been proposed to date. The work in [4] presented a known best linear trail with a correlation of 2^{-63} for 50-round CHAM-128/*. But they do not know whether there exists a better linear trail. Identifying the best variant

TABLE XIV
OPTIMAL DIFFERENTIAL TRAIL FOR 38-ROUND SAECHAM

Round	$\Delta X_r[0]$	$\Delta X_r[1]$	$\Delta X_r[2]$	$\Delta X_r[3]$	Probability
0	0x0800	0x0802	0x0401	0x0402	2^{-2}
1	0x0802	0x0401	0x0402	0x0200	2^{-3}
2	0x0401	0x0402	0x0200	0x0200	2^{-3}
3	0x0402	0x0200	0x0200	0x0100	2^{-2}
4	0x0200	0x0200	0x0100	0x0100	2^{-1}
5	0x0200	0x0100	0x0100	0x0000	2^{-1}
6	0x0100	0x0100	0x0000	0x0000	2^{-1}
7	0x0100	0x0000	0x0000	0x0000	2^0
8	0x0000	0x0000	0x0000	0x0000	2^0
9	0x0000	0x0000	0x8000	0x0000	2^0
10	0x0000	0x8000	0x0000	0x0000	2^{-1}
11	0x8000	0x0000	0x0000	0x0080	2^{-1}
12	0x0000	0x0000	0x0080	0x0040	2^0
13	0x0000	0x0080	0x0040	0x0000	2^0
14	0x0080	0x0040	0x0000	0x8000	2^{-1}
15	0x0040	0x0000	0x8000	0x4000	2^{-1}
16	0x0000	0x8000	0x4000	0x2000	2^{-1}
17	0x8000	0x4000	0x2000	0x0080	2^{-2}
18	0x4000	0x2000	0x0080	0x0080	2^{-2}
19	0x2000	0x0080	0x0080	0x0020	2^{-1}
20	0x0080	0x0080	0x0020	0x8010	2^0
21	0x0080	0x0020	0x8010	0x0000	2^{-2}
22	0x0020	0x8010	0x0000	0x2000	2^{-3}
23	0x8010	0x0000	0x2000	0x1080	2^{-2}
24	0x0000	0x2000	0x1080	0x0840	2^{-1}
25	0x2000	0x1080	0x0840	0x0020	2^{-2}
26	0x1080	0x0840	0x0020	0x8020	2^{-3}
27	0x0840	0x0020	0x8020	0x4008	2^{-2}
28	0x0020	0x8020	0x4008	0x0004	2^{-2}
29	0x8020	0x4008	0x0004	0x0080	2^{-4}
30	0x4008	0x0004	0x0080	0x0880	2^{-3}
31	0x0004	0x0080	0x0880	0x0440	2^{-1}
32	0x0080	0x0880	0x0440	0x8200	2^{-1}
33	0x0880	0x0440	0x8200	0x0008	2^{-2}
34	0x0440	0x8200	0x0008	0x0000	2^{-4}
35	0x8200	0x0008	0x0000	0x4082	2^{-3}
36	0x0008	0x0000	0x4082	0x0841	2^{-1}
37	0x0000	0x4082	0x0841	0x0800	2^{-2}
38	0x4082	0x0841	0x0800	0x8240	-

among these remains an open problem and is left as future work.

APPENDIX A TARGET MCUS

The 8-bit AVR target microcontroller is the ATmega128 from Atmel, which features 32 8-bit general purpose registers, 4KB of SRAM, 128KB of flash memory, and is capable of supporting a maximum clock speed of 16 MHz with low power consumption.

The 16-bit MSP430 target microcontroller is the MSP430F1611 from Texas Instruments, which is equipped with 12 16-bit general purpose registers, 10KB of SRAM,

48KB of flash memory, and supports a maximum clock speed of 8 MHz.

The 32-bit ARM Cortex-M3 target microcontroller is the STM32F103RB from STMicroelectronics, which features 20KB of SRAM, 128KB(128K ×8) of Flash memory, and is capable of supporting a maximum clock speed of 72 MHz.

The 32-bit ARM Cortex-M4 target microcontroller is the STM32F407VGT6 from STMicroelectronics, which features 192KB of SRAM, 1MB(1M ×8) of Flash memory, and is capable of supporting a maximum clock speed of 168 MHz with floating point unit (FPU) and DSP instructions.

APPENDIX B OPTIMAL TRAILS FOR SAECHAM

See Tables XIII and XIV.

REFERENCES

- [1] B. Koo, D. Roh, H. Kim, Y. Jung, D.-G. Lee, and D. Kwon, "Cham: A family of lightweight block ciphers for resource-constrained devices," in *Proc. Int. Conf. Inf. Security Cryptol.*, 2017, pp. 3–25.
- [2] D. Roh et al., "Revised version of block cipher Cham," in *Proc. 22nd Int. Conf. Inf. Security Cryptol.*, 2020, pp. 1–19.
- [3] M. Huang and L. Wang, "Automatic tool for searching for differential characteristics in ARX ciphers and applications," in *Proc. Int. Conf. Cryptol. India*, 2019, pp. 115–138.
- [4] D. Roh, "New linear trails and linear hulls of Cham," *IEEE Access*, vol. 12, pp. 103839–103847, 2024.
- [5] M. Huang, Z. Xu, and L. Wang, "On the probability and automatic search of rotational-XOR cryptanalysis on ARX ciphers," *Comput. J.*, vol. 65, no. 12, pp. 3062–3080, 2022.
- [6] A. Biryukov, J. S. Teh, and A. Udovenko, "Advancing the meet-in-the-filter technique: Applications to Cham and Katan," in *Proc. Int. Conf. Sel. Areas Cryptogr.* 2022, pp. 355–375.
- [7] M. Huang and L. Wang, "Automatic search for the linear (Hull) characteristics of ARX ciphers: Applied to SPECK, SPARX, Chaskey, and Cham-64," *Security Commun. Netw.*, vol. 2020, no. 1, 2020, Art. no. 4898612.
- [8] D. Hong, J.-K. Lee, D.-C. Kim, D. Kwon, K. H. Ryu, and D.-G. Lee, "LEA: A 128-bit block cipher for fast encryption on common processors," in *Proc. 14th Int. Workshop Inf. Security Appl.*, 2014, pp. 3–27.
- [9] R. Beaulieu, D. Shors, J. Smith, S. Treatman-Clark, B. Weeks, and L. Wingers, "The Simon and speck lightweight block ciphers," in *Proc. 52nd Annu. Design Autom. Conf.*, 2015, pp. 1–6.
- [10] S. Gueron, A. Jha, and M. Nandi, "COMET: Counter mode encryption with authentication tag," *Proc. 2nd Round Candidate NIST LWC Competition*, 2019, pp. 1–14.
- [11] H. Kwon, K. Jang, J. Park, and H. Seo, "High-speed implementation to Cham-64/128 counter mode with round key pre-load technique," *J. Korea Inst. Inf. Security Cryptol.*, vol. 30, no. 6, pp. 1217–1223, 2020.
- [12] S. Banik, S. K. Pandey, T. Peyrin, Y. Sasaki, S. M. Sim, and Y. Todo, "Gift: A small present: Towards reaching the limit of lightweight encryption," in *Proc. 19th Int. Conf. Cryptograph. Hardw. Embedded Syst.*, 2017, pp. 321–345.
- [13] L. Sun, B. Preneel, W. Wang, and M. Wang, "A greater gift: Strengthening gift against statistical cryptanalysis," in *Proc. Annu. Int. Conf. Theory Appl. Cryptograph. Techn.*, 2022, pp. 115–144.
- [14] M. Shin et al., "Efficient Cham-like structures on general-purpose processors with changing order of operations," *J. Korea Inst. Inf. Security Cryptol.*, vol. 34, no. 4, pp. 629–639, 2024.
- [15] E. Biham and A. Shamir, "Differential cryptanalysis of des-like cryptosystems," *J. Cryptol.*, vol. 4, pp. 3–72, Jan. 1991.
- [16] X. Lai, J. L. Massey, and S. Murphy, "Markov ciphers and differential cryptanalysis," in *Proc. Workshop Theory Appl. Cryptograph. Techn.*, 1991, pp. 17–38.
- [17] H. Lipmaa and S. Moriai, "Efficient algorithms for computing differential properties of addition," in *Proc. 8th Int. Workshop Fast Softw. Encrypt.*, 2002, pp. 336–350.
- [18] M. Matsui, "Linear cryptanalysis method for des cipher," in *Proc. Workshop Theory Appl. Cryptograph. Techn.*, 1993, pp. 386–397.
- [19] E. Schulte-Geers, "On CCZ-equivalence of addition mod 2^n ," *Designs, Codes Cryptogr.*, vol. 66, pp. 111–127, Jan. 2013.
- [20] M. Soos, K. Nohl, and C. Castelluccia, "Extending sat solvers to cryptographic problems," in *Proc. Int. Conf. Theory Appl. Satis. Testing*, 2009, pp. 244–257.
- [21] V. Ganesh and D. L. Dill, "A decision procedure for bit-vectors and arrays," in *Proc. 19th Int. Conf. Comput. Aided Verification*, 2007, pp. 519–531.
- [22] N. Mouha and B. Preneel, "Towards finding optimal differential characteristics for ARX: Application to salsa20," Cryptology ePrint Archive, IACR, Bellevue, WA, USA, Rep. 2013/328, 2013.
- [23] L. Song, Z. Huang, and Q. Yang, "Automatic differential analysis of ARX block ciphers with application to speck and LEA," in *Proc. Aust. Conf. Inf. Security Privacy*, 2016, pp. 379–394.
- [24] R. Ankele and S. Kölbl, "Mind the gap—A closer look at the security of block ciphers against differential cryptanalysis," in *Proc. Int. Conf. Sel. Areas Cryptogr.*, 2018, pp. 163–190.
- [25] S. A. Azimi, A. Ranea, M. Salmasizadeh, J. Mohajeri, M. R. Aref, and V. Rijmen, "A bit-vector differential model for the modular addition by a constant," in *Proc. 26th Int. Conf. Theory Appl. Cryptol. Inf. Security*, 2020, pp. 385–414.
- [26] R. Beaulieu, D. Shors, J. Smith, S. Treatman-Clark, B. Weeks, and L. Wingers, "The Simon and speck block ciphers on AVR 8-bit microcontrollers," in *Proc. 3rd Int. Workshop Lightweight Cryptography Security Privacy*, 2015, pp. 3–20.
- [27] D. Dinu, L. Perrin, A. Udovenko, V. Velichkov, J. Großschädl, and A. Biryukov, "Design strategies for ARX with provable bounds: SPARX and LAX," in *Proc. 22nd Int. Conf. Theory Appl. Cryptol. Inf. Security*, 2016, pp. 484–513.
- [28] D. Hong et al., "Hight: A new block cipher suitable for low-resource device," in *Proc. 8th Int. Workshop Cryptograph. Hardw. Embedded Syst.*, 2006, pp. 46–59.
- [29] H. Seo, K. An, and H. Kwon, "Compact LEA and hight implementations on 8-bit AVR and 16-bit MSP processors," in *Proc. 19th Int. Conf. Inf. Security Appl.*, 2019, pp. 253–265.



Myoungsu Shin received the B.S. degree in information technology and engineering from Jeonbuk National University, Jeonju, South Korea, in 2023. He is currently pursuing the M.S. degree with the Graduate School of Cyber Security, Korea University, Seoul, Republic of Korea.

His research focuses on symmetric cryptography.



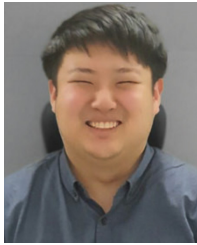
Hanbeom Shin received the B.S. degree in mathematics from the University of Seoul, in 2022 and the M.S. degree in information security from Korea University, Seoul, Republic of Korea, in 2024. He is currently pursuing the Ph.D. degree with the Graduate School of Cyber Security, Korea University.

His research focuses on symmetric cryptography.



Insung Kim received the B.S. degree in mathematics from University of Seoul, in 2022. He is currently pursuing the Ph.D. degree with the Graduate School of Cyber Security, Korea University, Seoul, Republic of Korea.

His research focuses on symmetric cryptography.



Sunyeop Kim received the B.S. degree in mathematics from Korea University, Seoul, Republic of Korea, in 2019, where he is currently pursuing the Ph.D. degree with the Graduate School of Cyber Security.

His research focuses on symmetric cryptography.



Jaechul Sung received the Ph.D. degree in mathematics from Korea University, Seoul, Republic of Korea, in 2002.

He worked as a Senior Researcher with the Korea Information Security Agency (KISA), Seoul, from July 2002 to January 2004. He is currently a Professor with the Department of Mathematics, University of Seoul, Seoul. His research interests include cryptography, symmetric cryptosystems, hash functions, and MACs.



Dongjae Lee received the Ph.D. degree in information security from Korea University, Seoul, Republic of Korea, in 2023.

From 2023 to 2024, he worked as a Postdoctoral Researcher with the Institute of Cyber Security and Privacy (ICSP), Korea University. He is currently a Professor with the Department of convergence security, Kangwon National University, Chuncheon, Republic of Korea. His research interests include symmetric cryptography and post-quantum cryptography.



Deukjo Hong received the B.S. and M.S. degrees in mathematics and Ph.D. degree in information security from Korea University, Seoul, Republic of Korea, in 1999, 2002, and 2006, respectively.

From 2007 to 2015, he worked for ETRI. He currently holds the position of Associate Professor with the Department of computer science and artificial intelligence, Jeonbuk National University, Jeonju, Republic of Korea. His research interest includes symmetric cryptography.



Seokhie Hong received the M.A. and Ph.D. degrees in mathematics from Korea University, Seoul, Republic of Korea, in 1997 and 2001, respectively.

He worked for Security Technologies Inc., Beijing, China, from 2000 to 2004. From 2004 to 2005, he worked as a Postdoctoral Researcher with COSIC, KU Leuven, Leuven, Belgium. Since 2005, he has been with Korea University, where he is now working with the Graduate School of Cyber Security. His specialty lies in the area of information security,

and his research interests include the design and analysis of symmetric-key cryptosystems, public-key cryptosystems, and side channel analysis.